

Le variabili in Java

In Java, una variabile è un'area di memoria utilizzata per memorizzare un valore durante l'esecuzione di un programma.

Ogni variabile è composta da:

- un tipo (*type*) che definisce quali valori può contenere e quanto spazio occupa in memoria;
- un nome (*identificatore*) con cui viene richiamata nel codice.

All'interno delle classi vengono inoltre dichiarate anteponendo un modificatore di accesso (spiegato più avanti).

Le variabili possono contenere dati di tipo primitivo (come int, double, boolean, char) o riferimenti a oggetti (come String, Array, OggettiPersonali, ...).

Prima di poter essere utilizzate, le variabili devono essere dichiarate e, in molti casi, inizializzate.

Dichiarazione e inizializzazione

La **dichiarazione** di una variabile specifica il tipo e il nome:

`int eta;`

L'**inizializzazione** assegna un valore:

`eta = 25;`

Spesso **dichiarazione e inizializzazione** sono combinate:

`int eta = 25;`

Attenzione: non si può inizializzare una variabile che non è stata dichiarata.

Tipi di variabili in Java

- *Variabili locali*: dichiarate all'interno di un metodo o blocco. Visibili solo all'interno di quel contesto (banalmente nel blocco di codice compreso tra due parentesi graffe).
- *Variabili di istanza*: (spiegate più avanti) dichiarate all'interno di una classe, ma fuori dai metodi. Rappresentano lo stato dell'oggetto.
- *Variabili statiche* (di classe): (spiegate più avanti) dichiarate con la parola chiave *static*. Condivise tra tutte le istanze della classe (cioè gli oggetti stanziati tramite quella classe).

Regole per i nomi delle variabili

- Devono iniziare con una lettera, `_` o `$`.
- Possono contenere lettere, cifre, `_` e `$`.
- Non possono essere parole chiave riservate (es. int, class).

È buona pratica usare nomi significativi e in **lowerCamelCase** (es. nomeUtente), nel senso che **è fortemente consigliato** che le variabili abbiano la prima lettera in carattere minuscolo, mentre le successive parole che compongono la variabile inizino con la lettera maiuscola.

C'è però da menzionare anche un'altra pratica, molto comune in linguaggi come Pascal, Python e C++, che quindi **è possibile trovare** usata anche il programmi scritti in Java, la **pascal_case**, in cui i nomi delle variabili sono composti da parole in minuscolo, separate da trattini bassi.

Esempio: `variabile_con_trattino_basso`, `numero_di_studenti`, `nome_completo`.

Sono da menzionare anche le costanti (spiegate più avanti) che vengono nominate usando la **PASCAL_CASE**, con le parole tutte in maiuscolo.

I tipi primitivi in Java

I tipi primitivi in Java sono i fondamenti per la dichiarazione delle variabili e sono predefiniti dal linguaggio.

Sono otto in totale e **non sono oggetti**, ma tipi di dato elementari con dimensione e intervallo di valori fissi, indipendenti dalla piattaforma.

Categorie dei tipi primitivi

I tipi primitivi possono essere suddivisi in quattro categorie principali:

- Tipo logico: *boolean*
- Tipo carattere: *char*
- Tipi integrali (numeri interi): *byte*, *short*, *int*, *long*
- Tipi a virgola mobile (numeri reali): *float*, *double*

Dettagli dei tipi primitivi

Tipo	Dimensione	Intervallo di valori	Esempio
boolean	1 bit	true o false	boolean attivo = true ;
char	16 bit	Da \u0000 (0) a \uffff (65535)	char lettera = 'A';
byte	8 bit	Da -128 a 127	byte piccolo = 100;
short	16 bit	Da -32768 a 32767	short valore = 30000;
int	32 bit	Da -2147483648 a 2147483647	int numero = 100000;
long	64 bit	Da -9223372036854775808 a 9223372036854775807	long grande = 100000000000L;
float	32 bit	Fino a ~7 cifre decimali	float decimale = 3.14f;
double	64 bit	Fino a ~15 cifre decimali	double preciso = 3.141592653589793;

Valori di default

Quando una variabile primitiva è dichiarata come campo di una classe (non locale), assume un valore di default, che banalmente possiamo definire zero e che nel caso di un boolean equivale a false.

Osservazione: gli intervalli di esistenza dei valori integrali hanno una motivazione ben precisa. Sono gestiti in Java con la codifica in complemento a 2 (si rimanda in appendice per un eventuale approfondimento).

boolean

In Java, il tipo boolean è un tipo primitivo che può assumere solo due valori:

- true (vero)
- false (falso).

Viene usato per espressioni logiche e condizioni.

Esempio:

```
boolean attivo = true;  
if (attivo) {  
    System.out.println("Attivo");  
}
```

byte

È il tipo numerico più piccolo e viene spesso utilizzato per risparmiare memoria in array di dati o quando si lavora con flussi di byte (es. I/O).

char

In Java, il tipo char è un tipo primitivo che rappresenta un singolo carattere. I valori char sono racchiusi tra apici singoli e utilizzano la codifica Unicode (UTF-16), occupando 16 bit (2 byte) di memoria.

Esempio base:

```
char lettera = 'A';  
char cifra = '3';  
char simbolo = '@';
```

Valori Unicode

È possibile usare valori Unicode con il prefisso `\u`:

```
char pi = '\u03C0'; // π  
char cuore = '\u2665'; // ♥
```

Caratteri di escape

Alcuni caratteri speciali si inseriscono con escape (`\`); qui di seguito alcuni esempi:

```
char capo = '\n'; // carattere di andata a capo  
char tab = '\t'; // carattere di tabulazione  
char barra = '\\'; // carattere di barra inversa  
char apice = '\"'; // carattere di apice singolo
```

Intervallo

Da `\u0000` a `\uffff`, in notazione esadecimale.

Copre quasi tutti i caratteri europei, asiatici e simboli.

Compatibilità con altre codifiche di caratteri o versioni di Java

Per le stringhe e l'I/O, Java gestisce automaticamente le conversioni tra Unicode e altre codifiche (come ASCII, UTF-8, ISO-8859-1, ecc.), ma l'encoding predefinito dipende dalla piattaforma.

Per evitare errori (es. caratteri mostrati come `?`), è buona pratica specificare esplicitamente l'encoding durante la lettura/scrittura di testi.

(Si rimanda in appendice per un approfondimento su cosa sia ASCII, UTF-8, ecc)

int

È il tipo intero più comune e viene usato per operazioni matematiche, indici, contatori e altre variabili numeriche.

long

È il tipo intero più grande disponibile in Java e viene usato quando il range di int non è sufficiente.

La sua dichiarazione richiede il suffisso **L** o **l** (si preferisce L per chiarezza perché l può essere scambiato per 1).

```
long grandeNumero = 10000000000L;
```

float

È un numero in virgola mobile a 32 bit (singola precisione), conforme allo standard **IEEE 754**.

Può rappresentare valori approssimativamente compresi tra

$\pm 1.4 \times 10^{-45}$ e $\pm 3.4 \times 10^{38}$,

con una precisione di circa 6-7 cifre decimali.

La sua dichiarazione richiede il suffisso **F** o **f**

```
float numeroDecimale = 3.14f;
```

Viene usato principalmente per risparmiare memoria in grandi array di numeri decimali, ma non è adatto per calcoli precisi (es. valuta), dove si preferisce double.

double

Il tipo primitivo double in Java è un numero in virgola mobile a 64 bit (doppia precisione), conforme allo standard **IEEE 754**.

Offre un intervallo molto ampio e una precisione di circa 15-17 cifre decimali.

La sua dichiarazione non richiede il suffisso (**D** o **d**), che però resta opzionale

```
double pi = 3.141592653589793;  
double grande = 1.23e10; // notazione scientifica
```

Rappresentazione visiva dello spazio occupato dai tipi primitivi in memoria

boolean	1 bit	0
char	16 bit	0010011101100100
byte	8 bit	00101010
short	16 bit	0001000111000110
int	32 bit	0000000000000000000000001111111111
long	64 bit	0010111100001111000000000011111110101010101010101000000000000101
float	32 bit	01000000010010001111010111000011
double	64 bit	0100000110110001110111100111100001001010000000000000000000000000

i numeri all'interno delle barre sono usati solo come esempio, sebbene non siano stati inseriti a caso

Quando dichiariamo una variabile, a seconda del tipo primitivo a cui si riferisce, il compilatore farà in modo che venga allocato un determinato spazio in memoria (RAM / cache) per quella variabile, anche se questa valesse zero.

Quindi, 25 e -2147483648, dichiarati come int, occupano entrambi 32 bit.

Vien da sé immaginare che l'uso di tipi primitivi come short e byte quando è richiesto un intervallo (range) molto piccolo di numeri in cui operare, richieda non solo poco spazio di immagazzinamento, ma anche poco tempo per lo spostamento tra le varie locazioni di memoria; rendendo di fatto più veloci i calcoli effettuati dal processore.

Ci sono però molti casi in cui la scelta di un determinato tipo primitivo può generare problemi non solo di compilazione, che possono per lo meno essere riconosciuti dal compilatore, ma anche di run-time, cioè quei problemi che il compilatore non riconosce, presentandosi solo in fase di esecuzione. Sono quegli errori che solitamente vengono definiti *bug* e per cui la risoluzione alle volte può diventare piuttosto complessa.

Casting delle variabili

La parola *casting* in metallurgia si riferisce alla colata di un metallo fuso all'interno di uno stampo che gli conferirà la forma una volta solidificato.

Possiamo immaginare i tipi primitivi come degli stampi di infusione, ognuno con la sua forma (per float e double sono addirittura 3 forme in una) e un suo volume, e i valori delle variabili come un metallo da dovervi versare dentro.

Basandoci su questa immagine e sullo schema visivo proposto sopra, possiamo dire che un tipo int ha una forma uguale, ma un volume più piccolo (la metà) di long. Mentre ha una forma diversa pur avendo lo stesso volume di float.

Cosa vuol dire questo?

Sostanzialmente, possiamo immaginare che, prendendo un float e convertendolo in int, le informazioni di quel numero si andranno a modificare, cioè non avranno la stessa forma. Questo perché i numeri in virgola mobile (secondo lo standard IEEE 754), sono composti da ben 3 informazioni specifiche (bit del segno, mantissa ed esponente), mentre i numeri integrali sono scritti con una sola informazione specifica (complemento a due).

Esempio di casting corretto:

```
int numero = 45;
float floatNumero = (float) numero; // Risultato 45.0

double pi = 3.14159;
int intPi = (int) pi; // Risultato: 3
```

Esempio di casting errato:

```
double pi = 3.14159;
int intPi = pi; // Errore di compilazione!
```

Mettendo da parte i tipi non numerici (char e boolean) e sebbene non sia vietato fare il casting inverso, possiamo vedere lo schema proposto sopra come a una gerarchia di versamento (dall'alto verso il basso), "senza" che si perdano sostanziali informazioni.

Cosa succede se facciamo il casting inverso? Dipende.

Nel passaggio da virgola mobile a integrale si perdono i numeri decimali (possiamo immaginare una ghigliottina che trancia via tutto ciò che c'è a destra del punto, senza arrotondamenti di sorta), ma non è comunque detto che il numero in virgola mobile sia abbastanza piccolo da essere contenuto nel nuovo primitivo assegnato, il che può portare a errori imprevisti (generalmente di overflow o underflow).

Usiamo come esempio un caso particolare: valori fuori dall'intervallo

Se il numero in virgola mobile è troppo grande per essere rappresentato come intero (ad esempio, 2147483648.0 per un int), il risultato del casting dipende dal tipo di conversione e può portare a valori imprevisti.

```
double doppio = 2147483648.0; // Fuori dal range di int
int intero = (int) d;
System.out.println(i); // Stampa: -2147483648
```

Questo accade perché il valore eccede il massimo valore rappresentabile da un int (2147483647) e si verifica un overflow, con comportamento definito dalle regole di conversione IEEE 754 e dal complemento a due.

Ovviamente il problema si presenta anche qualora si volesse effettuare un casting da double a float oppure da un integrale più grande a uno più piccolo. Esempio:

```
long tipoLong = Long.MAX_VALUE; // 9223372036854775807
int tipoInt = (int) tipoLong;
System.out.println(tipoInt); // Stampa: -1
```

Questo accade perché Long.MAX_VALUE ha una rappresentazione binaria che inizia con 0 (bit più significativo segno positivo) seguito da sessantatre 1. Quando vengono presi solo i 32 bit meno significativi (tutti trentadue 1 partendo da destra nella rappresentazione big endian), in complemento a due questo corrisponde al valore -1. Si rimanda in appendice per un approfondimento.

Classi Wrapper dei tipi primitivi (accenno)

Una delle prime cose che è stata detta sui tipi primitivi è che non sono oggetti. E questo rimane sostanzialmente e immutabilmente vero. Tuttavia, per poter ampliare le possibilità di utilizzo dei primitivi come fossero oggetti, sono state create

delle classi particolari dette Wrapper e che possiedono lo stesso nome del dato primitivo ma con la particolarità di avere la prima lettera maiuscola. Nell'esempio precedente è infatti stata usata la variabile di istanza MAX_VALUE appartenente al Wrapper Long.

Gli Array in Java

Gli array in Java sono strutture dati che memorizzano un insieme di elementi dello stesso tipo, con dimensione fissa.

Caratteristica identificativa degli array sono le parentesi quadre '[' '']'.

Dichiarazione:

```
int [] numeri; // preferito
// oppure
int numeri [];
```

Creazione (allocazione):

Bisogna usare il comando **new** per creare l'array e specificarne la dimensione:

```
numeri = new int [5]; // array di 5 interi
```

Inizializzazione:

Si possono inizializzare gli elementi singolarmente

```
numeri [0] = 52;
numeri [1] = 2320;
```

o al momento della dichiarazione:

```
int [] numeri = {52, 2320, 730, 4750, 0};
```

Nota bene: è assolutamente possibile che un valore si ripeta più volte all'interno dello stesso Array.

Accesso agli elementi:

Gli Array partono con il primo elemento che ha indice 0 (**ZERO**).

```
int numeroSingolo = numeri [2];
System.out.println (numeroSingolo) // stampa 730
```

Per conoscere la lunghezza di un Array si può usare l'attributo length, il quale a sua volta usa come primitivo il tipo int.

```
double [] questoArray = {0.5, 5, 3.14, 92, 13.45, 0, 24, 150.8545, 16, 32, 0, 92};
System.out.println (questoArray.length); // stampa 12
// sebbene questoArray abbia 12 elementi, il suo ultimo elemento ha posizione 11
System.out.println (questoArray [11]); // stampa 92
// se invocassimo questoArray [12], otterremmo Index 12 out of bounds for length 12
```

Pertanto, dato un Array composto da N elementi, i suoi indici vanno da 0 a N-1.

Le Stringhe

Fino ad ora abbiamo chiesto a Java di effettuare varie stampe a schermo dei valori di nostro interesse senza chiederci più di tanto cosa stesse stampando effettivamente, nemmeno quando gli passavamo intere frasi comprese tra i doppi apici (ad esempio "questo insieme di parole").

Un computer (o meglio ancora, la sua CPU) non sa cosa siano le parole, al massimo può comprendere cosa sia un carattere (char). Per fare in modo quindi che esegua la stampa di intere frasi c'è bisogno che gli si passi un comando che gli faccia processare carattere per carattere il testo che ci interessa vedere stampato.

A questo servono le stringhe, che, essendo un insieme di char concatenati, in modo molto semplicistico possono sembrare degli Array di char (e in alcuni linguaggi, come C e Assembly, è anche così), ma che però in Java vengono definite tramite la classe **String**, che non è derivata dal tipo Array. Chiariamo la cosa con un esempio:

```
char [] hello = {'C', 'i', 'a', 'o', '!'};
// cosa succede se passo semplicemente l'Array alla funzione di stampa?
System.out.println(hello [0] + hello [1] + hello [2] + hello [3] + hello [4]);
// stampa a schermo 413

// cosa succede se assieme all'Array, dopo passo anche una carattere scritto tra doppi apici?
System.out.println(hello [0] + hello [1] + hello [2] + hello [3] + hello [4] + "!");
// stampa a schermo 413!

// cosa succede se invece "!" lo passo prima dell'Array?
System.out.println("!" + hello [0] + hello [1] + hello [2] + hello [3] + hello [4]);
// stampa a schermo !Ciao!

// se invece di "!", passo '!' assieme all'Array (prima o dopo è uguale)?
System.out.println('!' + hello [0] + hello [1] + hello [2] + hello [3] + hello [4]);
// stampa a schermo 446

// infine, cosa succede se passo solo l'Array?
System.out.println(hello);
// stampa a schermo Ciao!
```

Come è possibile notare, c'è una grossa differenza tra un String e un semplice Array di char. Questo perché il char possiede, per così dire, due nature: la prima è quella di numero e la seconda è di simbolo.

Quando è stato passato l'Array come somma dei suoi elementi, la funzione println lo ha riconosciuto come un insieme di numeri di 16 bit ciascuno e per questo li ha sommati e poi ha stampato il risultato dopo averlo convertito in String.

Cosa che è accaduta anche nel quarto caso.

Stesso si può dire del secondo caso, a cui però, tra la conversione e la stampa, ha concatenato la stringa "!".

Solo nel terzo e quinto caso c'è stata una conversione totale di ogni carattere in string con successiva concatenazione, perché, nel terzo caso, prima di tutto, è stata passata la stringa "!", mentre nel quinto caso è stato dato solo il riferimento all'Array di char e la funzione println, per sua impostazione interna, ha saputo riconoscere la richiesta di una stampa di stringa.

Ma le differenze non si fermano qui:

Ciò che è stato fatto con l'Array, cioè accedere a ogni suo singolo elemento, non è possibile per un String:

```
String saluto = "Ciao!";
System.out.println(saluto[0]);
```

produce l'errore

```
error: array required, but String found
System.out.println(saluto[0]);
```

Bisogna piuttosto usare il metodo `.charAt(indice)`:

```
String frase = "Ciao!";
char carattere = frase.charAt(2); // Restituisce 'a'
```

Anche a costo di ripetersi, **la sintassi con le parentesi quadre [] è riservata agli array, non alle stringhe**. Anche se una String contiene una sequenza di caratteri, è un oggetto della classe String, non un array di char.

String è immutabile, mentre gli array sono mutabili:

```
char [] arr = {'a', 'b'};  
arr [0] = 'x'; // adesso arr = {'x', 'b'}
```

```
String str = "ab"; // str non può essere modificata direttamente  
str = "xb"; // può solo essergli assegnata una nuova stringa  
// come poi si arrivi a definire/generare la nuova stringa è un altro discorso
```

Gli operatori in Java

Java è fornito di un ricco assortimento di operatori con cui poter operare sulle variabili. È possibile catalogarli in categorie, sebbene durante lo studio di questi non sia “categorico” procedere per categorie (ad esempio + con += e & con &&).

Possiamo pertanto riassumerli in:

- Operatori *Aritmetici*;
- Operatori di *Assegnamento*;
- Operatori di *Incremento e Decremento*;
- Operatori *Relazionali*;
- Operatori *Logici*;
- Operatori *Bitwise*;
- Operatore *Condizionale*;

A seconda che coinvolgano uno, due o tre operandi, vengono chiamati *unari*, *binari*, *ternari*.

Di seguito vengono riportati per categoria:

Operatori Aritmetici

Operatore	Descrizione	Tipologia	Esempio
+	Addizione	binario	a + b
-	Sottrazione	binario	a - b
*	Moltiplicazione	binario	a * b
/	Divisione	binario	a / b
%	Resto (modulo)	binario	a % b
+	Concatenazione Stringhe	binario	"a" + "b"
-	Negazione aritmetica	unario	-a

Operatori di Assegnamento

Operatore	Descrizione	Tipologia	Esempio
=	L'operatore base di assegnamento	binario	a = b
+=	Equivale a $\rightarrow a = a + b$	binario	a += b
-=	Equivale a $\rightarrow a = a - b$	binario	a -= b
*=	Equivale a $\rightarrow a = a * b$	binario	a *= b
/=	Equivale a $\rightarrow a = a / b$	binario	a /= b
%=	Equivale a $\rightarrow a = a \% b$	binario	a %= b
&=	Equivale a $\rightarrow a = a \& b$	binario	a &= b
=	Equivale a $\rightarrow a = a b$	binario	a = b

Operatori di Incremento e Decremento

Operatore	Descrizione	Tipologia	Esempio
<code>++</code> prefissa	Incrementa "prima di tutto"	unario	<code>++x</code>
postfissa <code>++</code>	Incrementa "dopo tutto"	unario	<code>x++</code>
<code>--</code> prefissa	Decrementa "prima di tutto"	unario	<code>--x</code>
postfissa <code>--</code>	Decrementa "dopo tutto"	unario	<code>x--</code>

Operatori Relazionali

Confrontano due valori e restituiscono un *boolean* (*true* o *false*).

Operatore	Descrizione	Tipologia	Esempio
<code>==</code>	Uguale a	binario	<code>a == b</code>
<code>!=</code>	Diverso da	binario	<code>a != b</code>
<code>></code>	Maggiore di	binario	<code>a > b</code>
<code><</code>	Minore di	binario	<code>a < b</code>
<code>>=</code>	Maggiore o uguale di	binario	<code>a >= b</code>
<code><=</code>	Minore o uguale di	binario	<code>a <= b</code>

Operatori Logici

Operano su valori booleani, quindi variabili che possono valere solo *true* o *false*.

Esempio: `a = true; b = false; c = true; ...`

Operatore	Descrizione	Tipologia	Esempio
<code>&&</code>	AND logico (corto circuito)	binario	<code>a && b</code>
<code> </code>	OR logico (corto circuito)	binario	<code>A b</code>
<code>!</code>	NOT logico	unario	<code>!a</code>

Operatori Bitwise

Agiscono sui singoli bit dei valori interi

Operatore	Descrizione	Tipologia	Esempio
<code>&</code>	AND bit a bit	binario	<code>5 & 3 → 101 & 011 = 001 → 1</code> <code>true & false → 1 & 0 = 0 → false</code>
<code> </code>	OR bit a bit	binario	<code>5 3 → 101 011 = 111 → 7</code> <code>false true → 0 1 = 1 → true</code>
<code>^</code>	XOR (OR) esclusivo	binario	<code>5 ^ 3 → 101 ^ 011 = 110 → 6</code> <code>false ^ true → 0 ^ 1 = 1 → true</code>
<code>~</code>	Complemento (NOT)	unario	<code>~5 → ~00000101 = 11111010 → -6</code> (in complemento a 2 in 8 bit)
<code><<</code>	Shift a sinistra	binario	<code>5 << 1 → 101 << 1 = 1010 → 10</code>
<code>>></code>	Shift a destra	binario	<code>-8 >> 1 → (mantenendo il bit di segno)</code> <code>→ -4 (per un complemento a 2 in 32 bit)</code>
<code>>>></code>	Shift a destra senza segno	binario	<code>-8 >>> 1 → 2147483644</code> (per un complemento a 2 di 32 bit)

Non esiste un operatore shift a sinistra senza segno perché lo shift a sinistra è già intrinsecamente senza segno.

NOTA BENE: Gli operatori `&`, `^`, `|` sono bitwise quando usati con tipi interi, logici con i boolean.

Operatore Condizionale (Ternario)

È l'unico operatore ternario in Java.

Sintassi: `condizione ? valore1 : valore2`

Esempio esplicativo:

```
int max = (a > b) ? a : b;
```

Equivale a dire

```
if (a > b) max = a;
else max = b;
```

Tabella delle precedenze degli operatori di Java

La tabella mostra le precedenze assegnate agli operatori Java con il relativo senso di azione. Gli operatori sono elencati in ordine di precedenza: più in alto un operatore appare nella tabella, più alta è la sua precedenza. In un'espressione vengono valutati prima gli operatori con precedenza relativa più alta. Gli operatori sulla stessa riga hanno la stessa precedenza.

Categoria	Operatori	Associatività
operatori postfissi	<code>[]</code> <code>.</code> <code>(params)</code> <code>expr++</code> <code>expr--</code>	Sinistra → Destra
operatori unari	<code>++expr</code> <code>--expr</code> <code>+expr</code> <code>-expr</code> <code>~</code> <code>!</code>	Destra ← Sinistra
istanze e cast	<code>new</code> <code>(type)expr</code>	Destra ← Sinistra
Aritmetici moltiplicativi	<code>*</code> <code>/</code> <code>%</code>	Sinistra → Destra
Aritmetici additivi	<code>+</code> <code>-</code> (e concatenazione di caratteri)	Sinistra → Destra
Shift	<code><<</code> <code>>></code> <code>>>></code>	Sinistra → Destra
di relazione	<code><</code> <code>></code> <code><=</code> <code>>=</code> <code>instanceof</code>	Sinistra → Destra
Uguaglianza	<code>==</code> <code>!=</code>	Sinistra → Destra
Bitwise AND	<code>&</code>	Sinistra → Destra
Bitwise XOR	<code>^</code>	Sinistra → Destra
Bitwise OR	<code> </code>	Sinistra → Destra
AND	<code>&&</code>	Sinistra → Destra
OR	<code> </code>	Sinistra → Destra
Condizionale	<code>? :</code>	Destra ← Sinistra
Assegnamento	<code>=</code> <code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code> <code>%=</code> <code>&=</code> <code>^=</code> <code> =</code> <code><<=</code> <code>>>=</code> <code>>>>=</code>	Destra ← Sinistra

Quando nella stessa espressione appaiono operatori con la stessa precedenza occorre stabilire quale viene valutato prima.

Le parentesi `()` possono sovrascrivere la precedenza: `(a + b) * c`

Cerchiamo di dare maggiore contesto a ogni operatore

Riportati in modo così sintetico, gli operatori possono indurre a svariate interpretazioni, solitamente nel migliore dei casi esatte, ma monche delle loro piene potenzialità (come nel caso di & e | con i rispettivi && e ||).

L'operatore base di assegnamento =

È il primo operatore a cui ci si approccia in assoluto durante lo studio della programmazione, senza il quale non è possibile inizializzare le variabili. Prende ciò che si trova alla sua destra e lo assegna a ciò che si trova a sinistra. Esempio:

```
int a, b;  
b = 5;  
a = b;  
b = 6;  
System.out.println("a = " + a + " b = " + b); // stampa a = 5 b = 6  
char c, d;  
c = 'r';  
d = 'e';  
System.out.println("Umberto I era un " + c + d); // stampa Umberto I era un re
```

Osservazione: all'interno della prima chiamata alla funzione di stampa è stata eseguita una concatenazione di stringhe, sebbene a e b siano dei tipi int. Questo perché println converte ogni variabile che gli diamo in un oggetto String prima di stampare.

Gli operatori aritmetici

Per praticità partiamo con un esempio:

```
int a = 5, b = 2;  
int somma = a + b; // 7  
int diff = a - b; // 3  
int prod = a * b; // 10  
int div = a / b; // 2 (divisione intera)  
int resto = a % b; // 1
```

Attenzione: con tipi interi (int), la divisione tronca la parte decimale. Per risultati in virgola mobile, usa double o float nella dichiarazione di almeno una.

```
float a = 5, b = 2;  
float div = a / b;  
System.out.println("Divisione: " + div); // stampa Divisione : 2.5
```

Attenzione: Per risultati in virgola mobile partendo da operandi int, bisogna eseguire il cast di almeno un operando, altrimenti il risultato della divisione resta “comunque” intero.

```
int a = 5, b = 2;  
float div = a / b; // Prima esegue una divisione intera e poi converte in float  
System.out.println("Divisione: " + div); // stampa Divisione : 2.0  
  
// ecco cosa accade se invece si esegue il cast prima dell'operazione  
float div = a / (float) b; // Prima converte b in float e poi esegue una divisione  
System.out.println("Divisione: " + div); // stampa Divisione : 2.5  
  
float div = (float) a / b; // Prima converte a in float e poi esegue una divisione  
System.out.println("Divisione: " + div); // stampa Divisione : 2.5
```

Attenzione: `(float) a / b` non è affatto equivalente a `(float) (a / b)`

```
int a = 5, b = 2;
float div = (float) (a / b); // Prima esegue una divisione intera e poi converte in float
System.out.println("Divisione: " + div); // stampa Divisione : 2.0
```

Gli operatori di assegnamento +=, -=, *=, /=, % =

Scrivere molte righe di codice, può portare effettivamente a una ricerca di brevità di scrittura, soprattutto quando le variabili tendono a ripetersi.

Questo genere di operatori nascono proprio per agevolare questa esigenza.

Senza ripetere ciò che è già stato mostrato nella tabella più su, possiamo passare ad altri esempi più esplicativi:

```
int a = 5;
int b = 6;
int c = 7;
a += b + c; // equivale a a = a + (b + c)
System.out.println(a); // stampa 18
a -= b + c; // equivale a a = a - (b + c)
System.out.println(a); // stampa -8
a *= b + c; // equivale a a = a * (b + c)
System.out.println(a); // stampa 65
// stesso discorso vale per a /= b + c e per a %= b + c
```

L'operatore di assegnamento quindi non smette di essere un operatore di assegnamento solo perché ha un altro operatore che lo accompagna.

Prima che quindi venga effettuata l'operazione in questione, deve prima essere processato tutto ciò che si trova a destra dell'operatore e solo a quel punto viene eseguita l'operazione con assegnamento che coinvolge la variabile a sinistra.

Gli operatori di incremento e decremento

Anche in questo caso ci troviamo con un operatore nato per abbreviare i tempi di scrittura del codice.

La sua funzione è quella di aumentare o diminuire di 1 il valore di una variabile.

È per questo motivo l'operatore prediletto nei contatori e nei cicli.

Spieghiamolo con qualche esempio:

```
int a = 7;
a++;
System.out.println(a); // stampa 8
a--;
System.out.println(a); // stampa 7
++a;
System.out.println(a); // stampa 8
--a;
System.out.println(a); // stampa 7
```

Vista in questo modo possiamo dire che

`a++` equivale a dire `a += 1`, cioè `a = a + 1` (stesso dicasi per `++a`) e che

`a--` equivale a dire `a -= 1`, cioè `a = a - 1`

E la risposta a questa intuizione è NI.

Sì, perché presa da sola fa soltanto questo.

No, perché in effetti non fa soltanto questo.

Come è possibile notare, c'è un motivo se viene chiamata postfissa o prefissa a seconda che l'operatore sia dopo o prima della variabile interessata.

Nella notazione postfissa l'operazione avviene dopo l'esecuzione di un comando, mentre nella notazione prefissa viene effettuata prima di un comando.

E ciò è perfettamente in linea con la

tavola delle precedenze perché, anche quando usiamo la postfissa, l'operazione avviene alla fine perché è all'inizio che è stata decisa l'esecuzione in fine.

```
System.out.println(a++); // stampa 7
System.out.println(a); // stampa 8
System.out.println(a--); // stampa 8
System.out.println(a); // stampa 7
System.out.println(++a); // stampa 8
System.out.println(a); // stampa 8
System.out.println(--a); // stampa 7
System.out.println(a); // stampa 7
```

Operatori relazionali

Gli operatori relazionali in Java sono utilizzati per confrontare due valori e restituire un risultato booleano (true o false) in base al risultato del confronto. Sono fondamentali nelle espressioni condizionali, come quelle usate nei costrutti if, while, for e altri.

Possono confrontare variabili di tipo numerico (int, double, ecc.):

```
int x = 5;
int y = 6;

System.out.println("x == y: " + (x == y)); // false
System.out.println("x != y: " + (x != y)); // true
System.out.println("x > y: " + (x > y)); // false
System.out.println("x < y: " + (x < y)); // true
System.out.println("x >= y: " + (x >= y)); // false
System.out.println("x <= y: " + (x <= y)); // true
```

anche char può usufruire di questi operatori, ma bisogna capire perché e, per farlo, ci avvaliamo dei caratteri standard ASCII, che qui per semplicità viene riportata con rappresentazione decimale piuttosto che la effettiva binaria.

```
// notare l'uso degli apici singoli
// userò solo l'operatore < per semplicità
System.out.println('a' < 'b'); //true
System.out.println('a' < 'A'); //false
System.out.println('A' < '1'); //false
System.out.println('1' < '!'); //false
System.out.println('A' < '['); //true
System.out.println('a' < '['); //false
System.out.println('1' < '?'); //true
System.out.println('A' < '?'); //false
System.out.println('a' < '{'); //true
```

Il risultato del confronto tra i singoli caratteri è definito dal loro valore in codice binario.

Semplificando molto, per il processore il carattere a “equivale” a 97, sebbene la questione sia un po’ più complessa di così.

Codice	Carattere	Codice	Carattere	Codice	Carattere	Codice	Carattere
0	NULL	32	Spazio	64	@	96	`
1	Start of Heading	33	!	65	A	97	a
2	Start of text	34	“	66	B	98	b
3	End of text	35	#	67	C	99	c
4	End of transmit	36	\$	68	D	100	d
5	Enquiry	37	%	69	E	101	e
6	Acknowledge	38	&	70	F	102	f
7	Audible Bell	39	'	71	G	103	g
8	Backspace	40	(	72	H	104	h
9	Tab orizzontale	41	)	73	I	105	i
10	Line feed	42	*	74	J	106	j
11	Tab verticale	43	+	75	K	107	k
12	Form feed	44	,	76	L	108	l
13	Carriage return	45	-	77	M	109	m
14	Shift out	46	.	78	N	110	n
15	Shift in	47	/	79	O	111	o
16	Data link escape	48	0	80	P	112	p
17	Device control 1	49	1	81	Q	113	q
18	Device control 2	50	2	82	R	114	r
19	Device control 3	51	3	83	S	115	s
20	Device control 4	52	4	84	T	116	t
21	Neg. Acknowledge	53	5	85	U	117	u
22	Synchronous idle	54	6	86	V	118	v
23	End trans. Block	55	7	87	W	119	w
24	Cancel	56	8	88	X	120	x
25	End of medium	57	9	89	Y	121	y
26	Substitution	58	:	90	Z	122	z
27	Escape	59	;	91	[	123	{
28	File separator	60	<	92	\	124	
29	Group separator	61	=	93	]	125	}
30	Record separator	62	>	94	^	126	~
31	Unit separator	63	?	95	_	127	Del

Non si possono usare <, >, <=, >= con **boolean**, ma **solo con == e !=**

```
System.out.println(true == true); //stampa true
System.out.println(true == false); //stampa false
System.out.println(true != false); //stampa true
System.out.println(false == false); //stampa true

System.out.println(true < false); //stampa errore
// error: bad operand types for binary operator '<'
```

Questo perché true e false non sono valori fantoccio come accade in altri linguaggi di programmazione, come ad esempio in C, in cui false equivale a zero e qualsiasi cosa diversa da zero equivale a true e quindi sono “permessi” gli altri confronti.

Attenzione: == vs =

Un errore comune tra i principianti è confondere l'operatore di uguaglianza (==) con l'operatore di assegnamento (=):

```
int x;  
if (x = 5) // si ferma qui con error: incompatible types: int cannot be converted to boolean  
    System.out.println("ok");  
// stesso accadrebbe anche se x fosse un boolean
```

Quello corretto è:

```
int x = 5;  
if (x == 5)  
    System.out.println("ok"); // stampa ok
```

Attenzione: non si possono comparare variabili non inizializzate

```
int x;  
if (x == 5) // si ferma qui con error: variable x might not have been initialized  
    System.out.println("ok");
```

Gli operatori relazionali hanno una precedenza inferiore agli operatori aritmetici. Ad esempio:

```
int a = 10, b = 9, c = 2;  
boolean risultato = a > b + c; // viene valutato come: a > (b + c) → 10 > 11 → false
```

Come possiamo vedere dalla tabella delle precedenze **<, >, <=, >=** hanno la stessa precedenza e si associano da sinistra a destra. Mentre **== e !=** hanno precedenza più bassa rispetto ai precedenti.

Per i tipi primitivi (come int, double), == confronta i valori.

Per gli oggetti (come String), == confronta i riferimenti in memoria, non il contenuto.

Per confrontare il contenuto, si usa il metodo .equals():

```
String s1 = "ciao";  
String s2 = "ciao";  
System.out.println(s1 == s2); // Potrebbe essere true (grazie a string pooling)  
System.out.println(s1.equals(s2)); // true: confronto corretto del contenuto
```

Gli operatori logici

Gli operatori logici in Java sono fondamentali per costruire espressioni booleane complesse e controllare il flusso del programma attraverso condizioni. Operano su valori booleani (true o false) e restituiscono un risultato booleano.

&& (AND logico condizionale)

Restituisce true solo se entrambe le espressioni sono true.

Utilizza la valutazione in cortocircuito (short-circuit):

se la prima condizione è false, la seconda non viene valutata.

```
boolean a = true;  
boolean b = false;  
System.out.println(a && b); // Stampa: false
```

A	B	A && B
VERO	VERO	VERO
VERO	FALSO	FALSO
FALSO	VERO	FALSO
FALSO	FALSO	FALSO

|| (OR logico condizionale)

Restituisce true se almeno una delle espressioni è true.

Anche qui c'è valutazione in cortocircuito:

se la prima è true, la seconda non viene controllata.

```
System.out.println(a || b); // Stampa: true
```

A	B	A B
VERO	VERO	VERO
VERO	FALSO	VERO
FALSO	VERO	VERO
FALSO	FALSO	FALSO

! (NOT logico)

Operatore unario che inverte il valore booleano.

```
System.out.println(!a); // Stampa: false
```

A	!A
VERO	FALSO
FALSO	VERO

Gli operatori bitwise

Gli operatori bitwise in Java permettono di manipolare direttamente i singoli bit dei valori interi (long, int, short, char, byte), caso particolare per &, | e ^ che possono operare anche su valori boolean. Sono utili in contesti di programmazione a basso livello, ottimizzazione, crittografia, gestione di flag o compressione dati.

Per continuità con ciò che è stato detto per gli operatori logici, trattiamo prima il caso particolare riferito all'applicazione su valori boolean.

& (AND logico completo)

Restituisce true solo se entrambi gli operandi sono true.

Valuta sempre entrambe le espressioni, anche se la prima è false.

```
boolean a = true;  
boolean b = false;  
System.out.println(a & b); // false
```

| (OR logico completo)

Restituisce true se almeno uno è true.

Valuta sempre entrambi, anche se il primo è true.

```
System.out.println(a | b); // true
```

Confronto tra && e & e tra || e |

Osservando la tabella delle precedenze, è possibile notare come sia possibile l'interazione tra questi operatori nella stessa espressione. Bisogna però come sempre stare molto attenti quando si mischiano gli operatori, per non incappare in strani comportamenti in fase di esecuzione.

```
boolean a = true, b = false, c = true
```

```
System.out.println((a && c) & (!b)); // true & true → true
```

```
System.out.println(a && c & !b); // true, ma non per gli stessi motivi della precedente
```

```
// infatti (guardare tabella precedenze)
```

```
// prima avviene      !b → true
```

```
// poi                c & true → true
```

```
// in fine            a && true → true
```

```
System.out.println((b || c) | (b & c)); // true | false → true
```

```
System.out.println(b || c | b & c); // anche qui true, ma non per gli stessi motivi della precedente
```

```
// prima avviene      b & c → false
```

```
// poi                c | false → true
```

```
// in fine            b || true → true
```

in questi casi si è potuto ottenere gli stessi risultati, ma non è assolutamente detto che scrivendo altre sequenze il risultato non cambi, proprio perché, andando a guardare più nel dettaglio, non si comportano in modo uguale.

```
String s = null;  
// Con && → sicuro (cortocircuito)  
if (s != null && s.length() > 0) {  
    System.out.println("Stringa non vuota");  
}  
// Con & → PERICOLO: valuta sempre entrambe → NullPointerException!  
// if (s != null & s.length() > 0) → ERRORE se s è null
```


^ (XOR logico, anche chiamato OR esclusivo)

Restituisce true solo se i due operandi booleani sono diversi, false se sono uguali.

Esempio:

```
boolean a = true, b = false;
```

```
System.out.println(a ^ b); // true (diversi)
System.out.println(a ^ a); // false (uguali)
System.out.println(b ^ b); // false (uguali)
```

A	B	A ^ B
VERO	VERO	FALSO
VERO	FALSO	VERO
FALSO	VERO	VERO
FALSO	FALSO	FALSO

Attenzione: XOR è utile solo tra due operandi per logica esclusiva.
È sbagliato usare ^ come operatore per dire “solo uno”.
Esempio:

```
boolean a = true, b = true, c = true;
System.out.println(a ^ b ^ c); // true (a^b=false, false^c=true) → non intuitivo!
```

Trattiamo ora brevemente il caso di utilizzo degli operatori bitwise con variabili numeriche. Usati in questo modo restituiscono un valore binario (e non una semplice risposta in boolean), che poi viene interpretata a seconda delle circostanze. Gli operatori bitwise in Java eseguono la comparazione bit a bit tra due numeri, partendo dal bit meno significativo in termini di elaborazione logica, ma l'ordine di scansione non è definito dal linguaggio in senso stretto poiché l'operazione è parallela a livello logico. Facciamo un esempio pratico:

```
12 = 1100    1100    1100
5  = 0101    0101    0101
    &        |        ^
    0100    1101    1001
cioè = 4      = 13     = 9
```

L'esempio accanto va letto in questo modo:

12 vale 1100 in binario (4bit senza segno)

5 vale 0101 in binario (4bit senza segno)

Usando gli operatori come ci hanno insegnato a fare le addizioni a scuola, eseguiamo i confronti e poi leggiamo il risultato. &, |, ^ hanno le stesse regole imposte sopra, con vero = 1 e falso = 0.

L'operatore ~ funziona con il !, invertendo gli 1 in 0 e viceversa. Si dice che complementa i bit. Esempio:

```
byte a = 60; // 0011 1100 in complemento a due
System.out.println(~a); // stampa -61 (1100 0011 in complemento a due)
```

Anche per gli operatori di spostamento si fa prima a portare un esempio:

```
int x = 60; // 0011 1100
System.out.println(x << 2); // 240 (1111 0000) – moltiplica per 4
System.out.println(x >> 2); // 15 (0000 1111) – divide per 4 (arrotondato)
System.out.println(x >>> 2); // 15 (0000 1111) – stesso risultato per numeri positivi
```

Tutto ciò ci è di una certa utilità? Per la maggior parte del tempo no. Senza entrare nello specifico, solitamente gli operatori di bitwise vengono utilizzati in crittografia o per la manipolazione delle immagini (tipo l'applicazione di un filtro bianco e nero).

Gli operatori composti &=, |=, ^=, <<=, >>=, >>>=

Non c'è molto altro da dire in più su questi operatori, se non fare presente che esistono, visto che sostanzialmente usano le stesse regole degli altri operatori di assegnamento.